

Tactical Detection

The key problem of modern SOC teams is to rely too much on default rulesets that have been enabled. Unfortunately, the common/default settings won't cut it and must be tailored to the company.

Unique Threat Intelligence

Unique IOCs of previous intrusions are good examples of Threat Intel as they're traces of the specific adversary that your environment has already faced. The inclusion of these IOCs in your detection mechanism will help spot re-intrusion of that specific adversary immediately.

File Name	Path	File Size	Description
bad3xe69.exe	C:\Downloads\bad3xe69.exe	13619KB	Downloaded from the internet. Posing as an installer for an application that needs an update.

DNS/IP Address	Description
bad3xe69connection.io	Domain of the source of the malicious executable bad3xe69.exe
kind4bad.com	Suspicious domain related to bad3xe69connection.io
nic3connection.io	Suspicious domain related to bad3xe69connection.io

In the screenshot above, the direct indicator found by the authors of the documentation is actually just the bad3xe69connection[.]io; however, upon further inspection of the malicious domain, they were able to conclude that two other malicious domains should be recorded as IOCs due to their association with the original malicious domain.

To maximize our efficiency, we will transform these IOCs into detection rules in vendor-agnostic format using Sigma.

What is Sigma?

Sigma is an open-source generic signature language developed to describe log events in a structured format. This allows for quick sharing of detection methods by security analysts.

```

title: Executable Download from Suspicious Domains
status: test
description: Detects download of executable types from hosts found in the IOC spreadsheet
author: Mokmokmok
date: 2022/08/23
modified: 2022/08/23
logsource:
  category: proxy
detection:
  selection:
    c-uri-extension:
      - 'exe'
    r-dns:
      - 'bad3xe69connection.io'
      - 'kind4bad.com'
      - 'nic3connection.io'
  condition: selection
fields:
  - c-uri
falsepositives:
  - Unkown
level: medium

```

Picture 1 : Sigma is standardization of detection rules and can be used with **any SIEM** (ELK, Splunk, Sentinel etc.)
 Proxy server = middleman between users and the internet, in the company it helps with monitoring and blocking malicious websites and detecting malware
 Product-agnostic = Not connected to one vendor

This Sigma rule that we have constructed from the IOCs presented above is very simple and straightforward, yet the additional layer of detection that it gives the organization is invaluable. In the grander scheme of things, these layers work together to give your analysts the visibility that they need to spot bad actors before it's too late.

Remember that the bad guys need to circumvent all our defenses in order to get to their objectives, but we only need them to fail one layer of detection to have an idea that they're there.

Publicly Generated IOCs

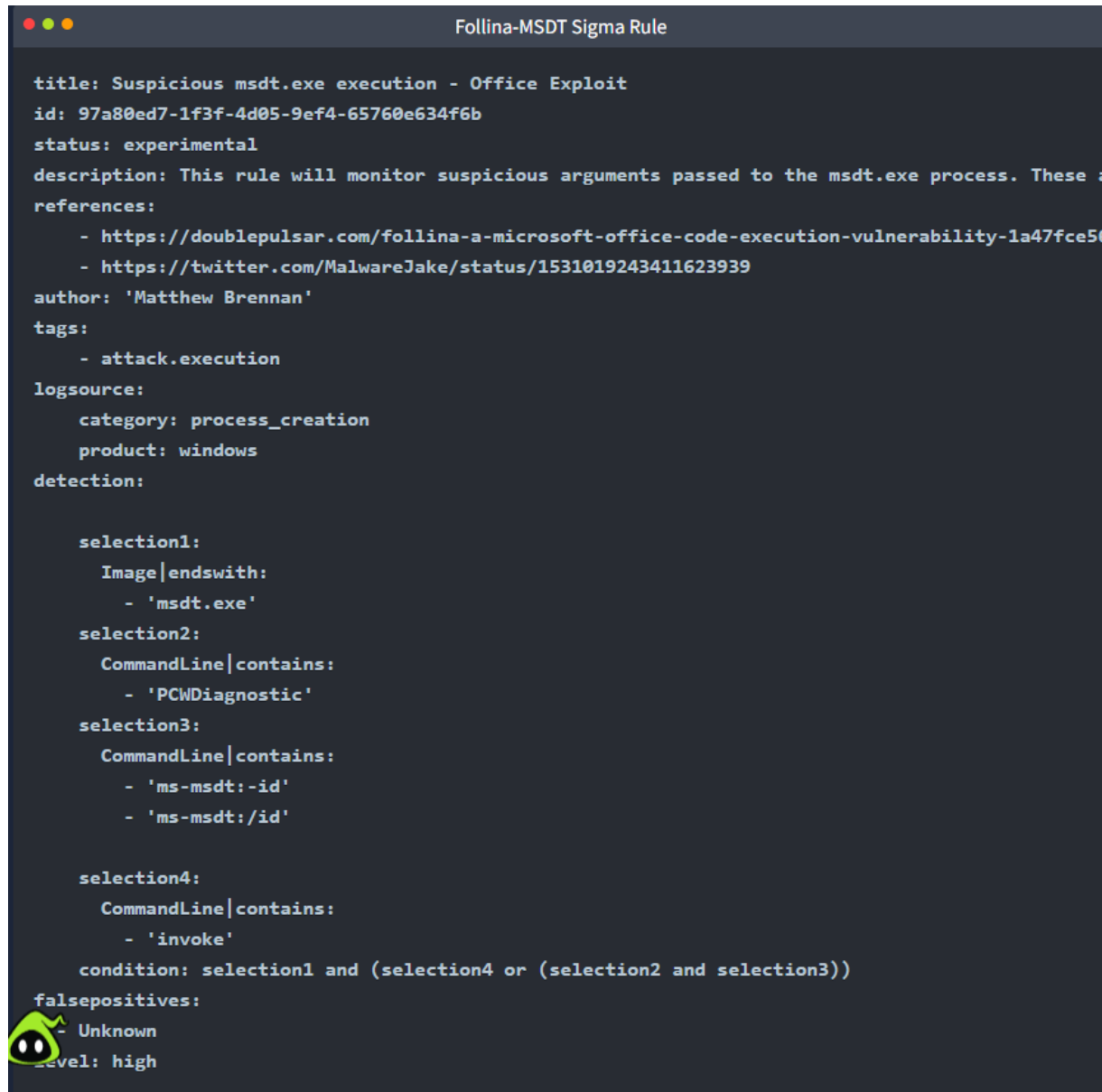
We don't have to be able to experience everything in order to learn from something – we can learn from other people's experience or research or learnings. For example, research is being done by community and they release public IOCs. These public IOCs are then transformed into usable mechanisms to detect bad things in the environment.

So we can basically download a repository with Sigma rules from a community and plug it directly into our SIEM for immediate value or edit it and then plug it.

PRACTICAL

Write a detection rule for these two / transform these publicly generated IOCs into usable alerts for use in the Elastic Stack and Splunk. For our purposes, we will be using Uncoder to help with the transformation of these sigma rules.

Uncoder is a nice tool that helps convert sigma rules to queries that can be immediately used within a SIEM of our choice.



```
title: Suspicious msdt.exe execution - Office Exploit
id: 97a80ed7-1f3f-4d05-9ef4-65760e634f6b
status: experimental
description: This rule will monitor suspicious arguments passed to the msdt.exe process. These :
references:
  - https://doublepulsar.com/follina-a-microsoft-office-code-execution-vulnerability-1a47fce50
  - https://twitter.com/MalwareJake/status/1531019243411623939
author: 'Matthew Brennan'
tags:
  - attack.execution
logsource:
  category: process_creation
  product: windows
detection:
  selection1:
    Image|endswith:
      - 'msdt.exe'
  selection2:
    CommandLine|contains:
      - 'PCWDiagnostic'
  selection3:
    CommandLine|contains:
      - 'ms-msdt:-id'
      - 'ms-msdt:/id'
  selection4:
    CommandLine|contains:
      - 'invoke'
  condition: selection1 and (selection4 or (selection2 and selection3))
falsepositives:
  - Unknown
level: high
```

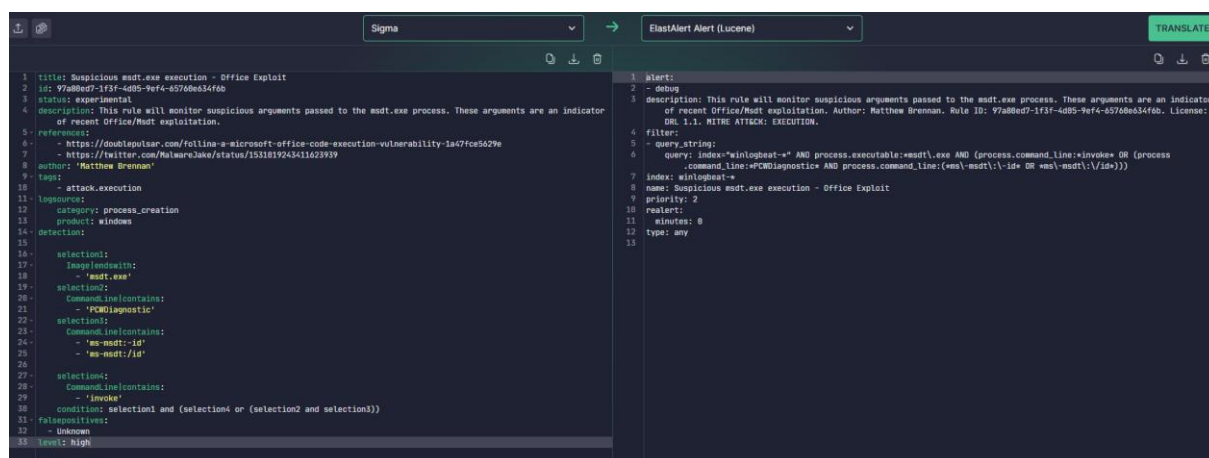
Picture 2 : A fairly recent 0-day vulnerability, Follina-MSDT, has a publicly available sigma rule developed by Matthew Brennan (<https://www.huntress.com/blog/microsoft-office-remote-code-execution-follina-msdt-bug>)

I have also found similar Sigma rule in Github, that could be also used for our rule making process (from https://github.com/SigmaHQ/sigma/blob/master/rules/windows/process_creation/proc_creation_win_msdt_susp_parent.yml).

```

title: Suspicious MSDT Parent Process
id: 7a74da6b-ea76-47db-92cc-874ad90df734
status: test
description: Detects msdt.exe executed by a suspicious parent as seen in CVE-2022-30190 / Follina exploitation
references:
  - https://twitter.com/nao_sec/status/1530196847679401984
  - https://app.any.run/tasks/713f05d2-fe78-4b9d-a744-f7c133e3fafb/
author: Nextron Systems
date: 2022-06-01
modified: 2023-02-06
tags:
  - attack.stealth
  - attack.t1036
  - attack.t1218
logsource:
  category: process_creation
  product: windows
detection:
  selection_parent:
    ParentImage|endswith:
      - '\cmd.exe'
      - '\cscript.exe'
      - '\mshta.exe'
      - '\powershell.exe'
      - '\pwsh.exe'
      - '\regsvr32.exe'
      - '\rundll32.exe'
      - '\schtasks.exe'
      - '\wmic.exe'
      - '\wscript.exe'
      - '\wsl.exe'
      # Note: office applications are covered by: 438025f9-5856-4663-83f7-52f878a70a50
  selection_msdt:
    - Image|endswith: '\msdt.exe'
    - OriginalFileName: 'msdt.exe'
  condition: all of selection_*
falsepositives:
  - Unknown
level: high

```



At this point we open Uncoder.io and paste our sigma rule and then choose what SIEM we wanna translate it into, we have chosen ElastAlert. (We can choose from almost 30 different SIEMs including Splunk, ElasticStack etc.)

Keep in mind that there is no guarantee that the transformed Sigma rules will work perfectly straight out of Uncoder. In order to be production ready, we need to do a lot of testing and fine-tuning. What Uncoder essentially offers is a generic blueprint – it is up to us to further improve it.

PRACTICAL

One way to create immediate impact is to leverage very specific data to your advantage. Depending on the organization, more often than not, there exist pieces of data that not everyone is entitled to have access to. Usually, controls are set by the IT team to limit these accesses.

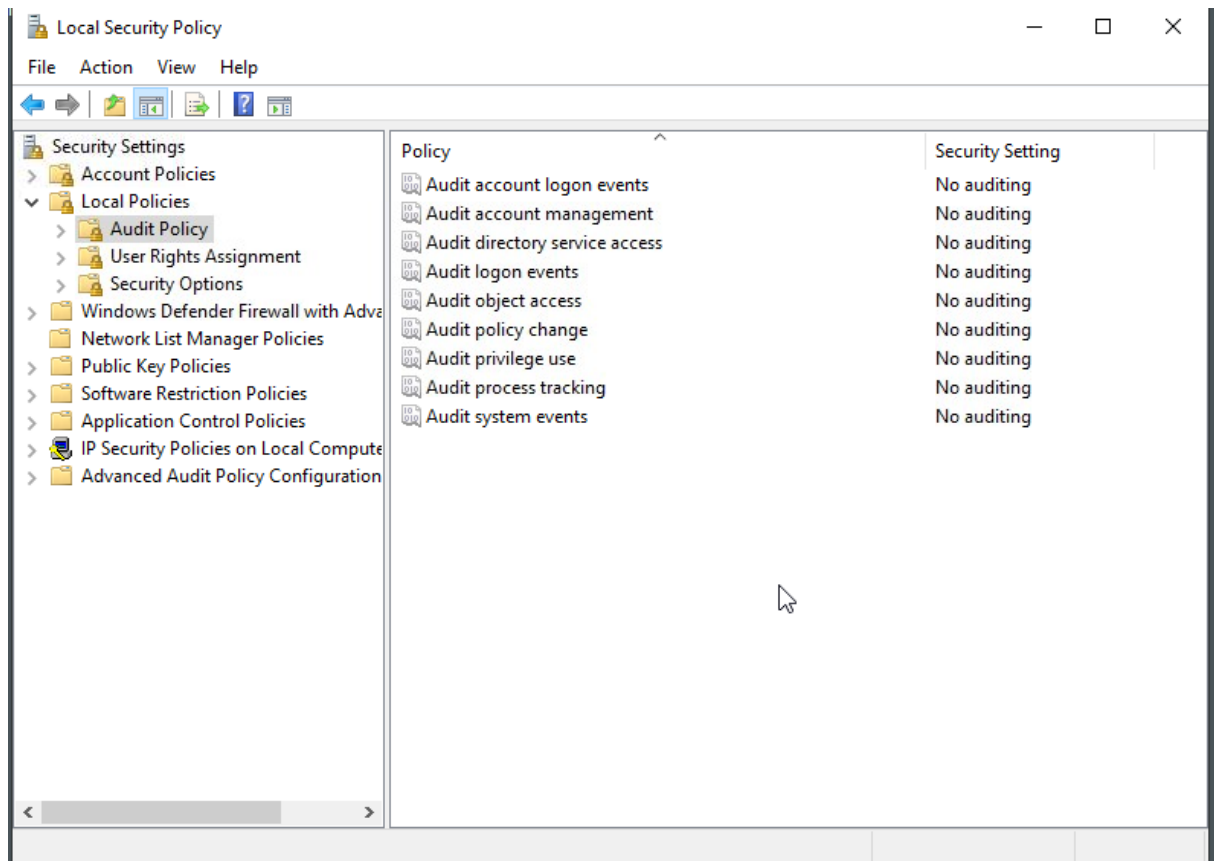
For example, suppose there are ultra-sensitive files your organization intends to keep secret, such as a hidden treasure map. In that case, you can set alert for instances that the said map has been accessed, edited or deleted etc. and then filter out the ones allowed access to make detections more actionable.

Tripwires are a great way to supplement the detection mechanisms that you already have in place. It's a way to cover "unknown unknowns" and even study an adversary. The most common tripwires are Honeypots and Hidden Files and Folders.

The way honeypots work is that they server no legitimate business purpose so any activity concerning them should raise immediate red flags. Hidden files and folders, on the other hand, are not meant to even be seen by normal end users, and so it works best when dealing with crawlers like worms making them particularly effective to detect intrusions.

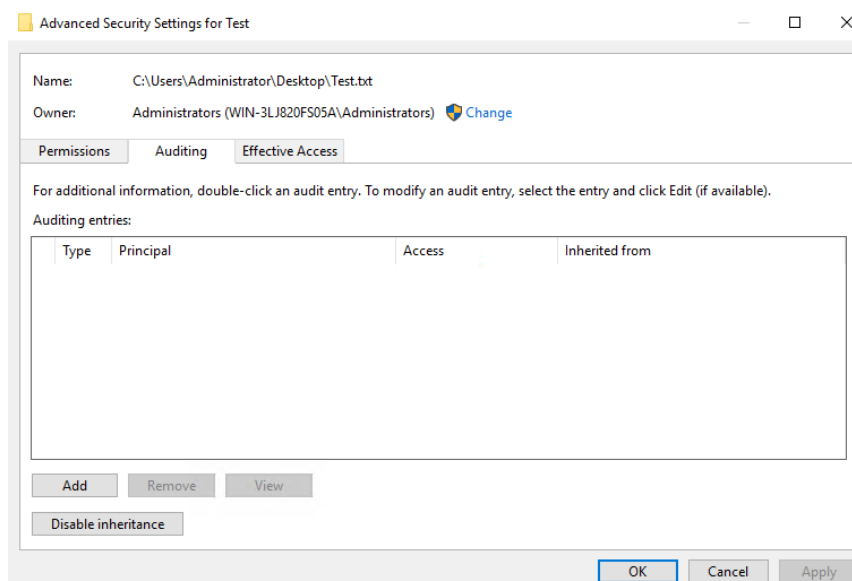
Setting up a basic tripwire

In Local security policy application, we will navigate ourselves to Security Settings > Local Policies > Audit Policy.

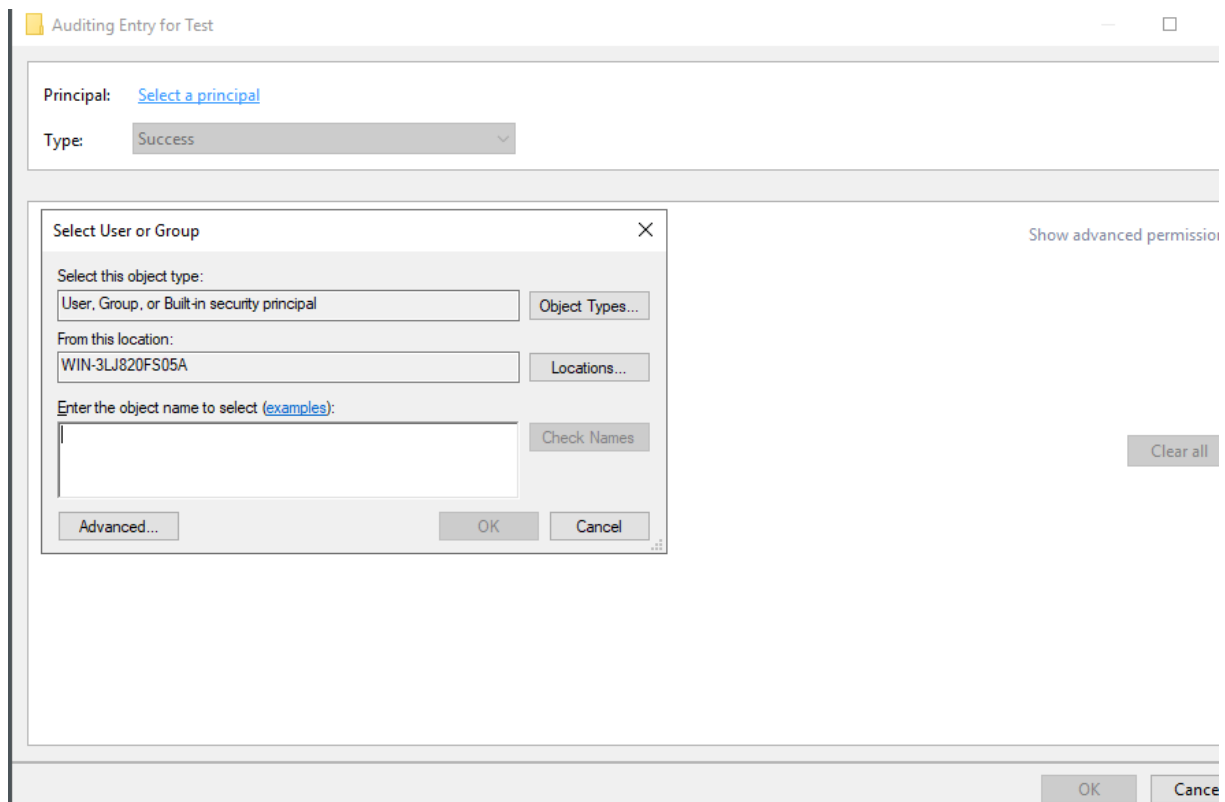


We open the Audit object access policy, tick the boxes beside Success and Failure, click Apply and finally click OK. This entails that all access attempts will be logged, regardless of whether it succeeded.

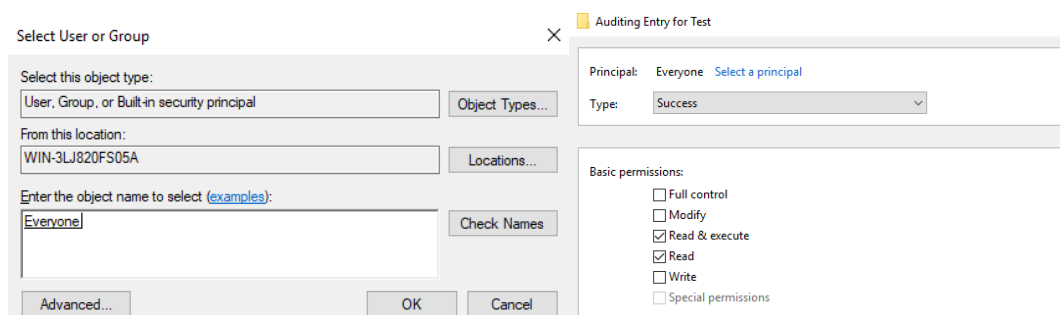
Now, we must also specify the actual file or folder that we intend to monitor so we will create a fresh file wherein our tripwire setup can be implemented and consequently observed. Right-click anywhere on the desktop > New > Text document. Then right click the document > Properties > Security > Advanced > Auditing



This is where we will create and specify an audit entry for our Test document. Click on Add > Select a principal.



In this particular window we can specify which users or groups we want to be alerted on whenever the access our Test document. We have a choice to be very granular, but for our purposes, we intend to be general and so, write “Everyone”, then press Enter.



Afterwards we can also specify all the activities on our Test document we want to audit. Then we just need to click OK and Apply.

Now the setup is complete and auditing is active for our Test document. Anyone who accesses will be logged and its details will be recorded in the Security event log with an Event ID 4663, this event ID can be filtered and furnished into alerts that would immediately tell the SOC analysts of tripwires being activated.

When we try to view the Test file, we will immediately get logs and see them in FullEventLogView:

FullEventLogView									
File Edit View Options Help									
Event Time	Record ID	Event ID	Level	Channel	Provider	Description	Opcode	Task Category	
6/18/2026 4:57:3...	9924	4690	Undefined	Security	Microsoft-Windows-Security...	An attempt was made to duplicate a handl...		Hai	
6/18/2026 4:57:3...	9925	4658	Undefined	Security	Microsoft-Windows-Security...	The handle to an object was closed. Subject...		Rer	
6/18/2026 4:57:3...	9926	4656	Undefined	Security	Microsoft-Windows-Security...	A handle to an object was requested. Subje...		Rer	
6/18/2026 4:57:3...	9927	4663	Undefined	Security	Microsoft-Windows-Security...	An attempt was made to access an object. ...		Rer	
6/18/2026 4:57:3...	9928	4658	Undefined	Security	Microsoft-Windows-Security...	The handle to an object was closed. Subject...		Rer	
6/18/2026 4:57:3...	9929	4690	Undefined	Security	Microsoft-Windows-Security...	An attempt was made to duplicate a handl...		Hai	
6/18/2026 4:57:3...	9930	4658	Undefined	Security	Microsoft-Windows-Security...	The handle to an object was closed. Subject...		Rer	
6/18/2026 4:57:3...	9931	4656	Undefined	Security	Microsoft-Windows-Security...	A handle to an object was requested. Subje...		Rer	
6/18/2026 4:57:3...	9932	4663	Undefined	Security	Microsoft-Windows-Security...	An attempt was made to access an object. ...		Rer	
6/18/2026 4:57:3...	9933	4663	Undefined	Security	Microsoft-Windows-Security...	An attempt was made to access an object. ...		Rer	
6/18/2026 4:57:3...	9934	4658	Undefined	Security	Microsoft-Windows-Security...	The handle to an object was closed. Subject...		Rer	
6/18/2026 4:57:3...	9935	4690	Undefined	Security	Microsoft-Windows-Security...	An attempt was made to duplicate a handl...		Hai	
6/18/2026 4:57:3...	9936	4658	Undefined	Security	Microsoft-Windows-Security...	The handle to an object was closed. Subject...		Rer	
6/18/2026 4:57:3...	9937	4656	Undefined	Security	Microsoft-Windows-Security...	A handle to an object was requested. Subje...		Rer	
6/18/2026 4:57:3...	9938	4663	Undefined	Security	Microsoft-Windows-Security...	An attempt was made to access an object. ...		Rer	
6/18/2026 4:57:3...	9939	4658	Undefined	Security	Microsoft-Windows-Security...	The handle to an object was closed. Subject...		Rer	
6/18/2026 4:57:3...	9940	4690	Undefined	Security	Microsoft-Windows-Security...	An attempt was made to duplicate a handle to an object.		Hai	
6/18/2026 4:57:3...	9941	4658	Undefined	Security	Microsoft-Windows-Security...	The handle to an object was closed. Subject...		Rer	
6/18/2026 4:57:3...	9942	4656	Undefined	Security	Microsoft-Windows-Security...	A handle to an object was requested. Subje...		Rer	
6/18/2026 4:57:3...	9943	4658	Undefined	Security	Microsoft-Windows-Security...	The handle to an object was closed. Subject...		Rer	
6/18/2026 4:57:4...	9944	4690	Undefined	Security	Microsoft-Windows-Security...	An attempt was made to duplicate a handl...		Hai	
6/18/2026 4:57:4...	9945	4658	Undefined	Security	Microsoft-Windows-Security...	The handle to an object was closed. Subject...		Rer	
6/18/2026 4:57:4...	9946	4656	Undefined	Security	Microsoft-Windows-Security...	A handle to an object was requested. Subje...		Rer	
6/18/2026 4:57:4...	9947	4658	Undefined	Security	Microsoft-Windows-Security...	The handle to an object was closed. Subje...		Rer	
An attempt was made to access an object.									
Subject:									
Security ID:	S-1-5-21-2936880785-3464050833-968612378-500								
Account Name:	Administrator								
Account Domain:	WIN-3LJ820FS05A								
Logon ID:	0x47B79								
Object:									
Object Server:	Security								
Object Type:	File								
Object Name:	C:\Users\Administrator\Desktop\Test.txt								
Handle ID:	0x1ed0								
Resource Attributes:	S:AI								
Process Information:									
Process ID:	0x1440								
Process Name:	C:\Windows\explorer.exe								

The process name is explorer.exe, because we run this room on the Tryhackme's attackbox virtual machine.

For better understanding of Object Access Events:

<https://www.ultimatewindowssecurity.com/securitylog/book/page.aspx?spid=chapter7>

The idea behind tactical detection is simple. If we want to see how our defenses fare hold against an attack, understand how a certain vulnerability works and what it looks like on the logs, or we simply want to know the extent of our visibility in our environment – we simulate an attack and then check the results.